3. Finite State Machines

Introduction

An FSM is used to model systems that transit among a finite number of internal states. In practice, the main application of an FSM is to act as the controller of a large digital system. If the output is only a function of state, the machine is known as Moore. If the output of the machine is a function of state and external input it is known as Mealy.
In the figure 1 both models are represented by a block diagram.

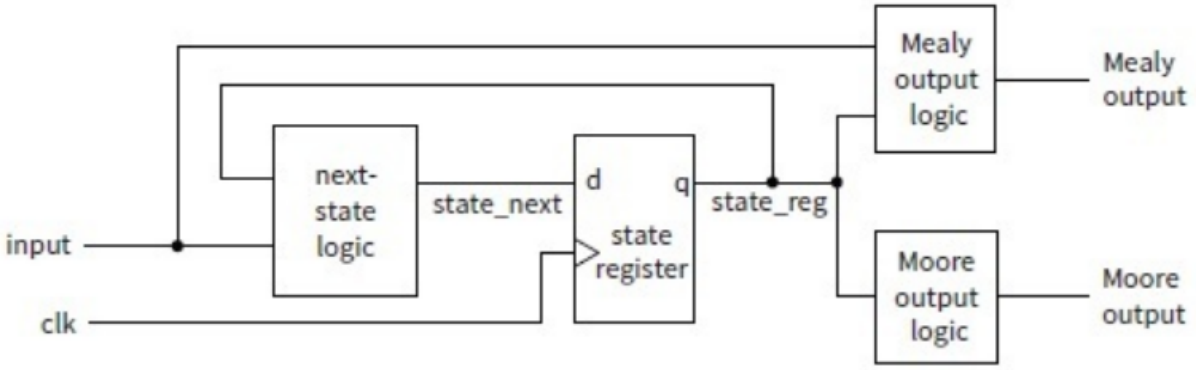


Fig. 1: Mealy and Moore block diagram

Mealy and Moore block diagram

Code development

An FSM can be specified by **state diagram** or an **ASM chart** (Algorithmic State Machine chart). The ASM chart representation is more descriptive for applications with complex transition conditions and actions.
The procedure for encoding an FSM consists first of separating the state register and then deriving the code for combinational next state logic and the output.
To make coding easier, we can follow the process shown below.

```
//module definition

//fsm state type
typedef enum {q0,q1,q2,..} state_type;

//signal declaration
state_type state_reg, state_next;
```

```
//state register
always_ff @(posedge clk, posedge reset) //posedge means positive edge
    if (reset)
        ...
    else
        state_reg = state_next;

//next-state logic
always_comb
    case (state_reg)
        q0: if (input)
                state_next = ...
            else
                state_next = ...
        ...
        default: state_next = ...
    endcase
//Moore output logic
assign z1 ...

//Mealy output logic
assign z2 ...
```

Exercise: Mealy and Moore machines.
Design and model (with JFLAP) a machine (FSM) that detects the pattern "01100110" in a series of bits given as input. When the pattern is found, it will send a logical "1" to the output. The same FSM should be modeled with the two classical machines: Mealy and Moore.
Code and simulate to analyze the results.

Exercise: Frequency divider.
A counter can be used to divide the frequency *f* of a clock, where the output frequency q(*i*) is:

$$f_i = \frac{f}{2^{(i+1)}}$$

The **Basys 3** board includes a 100 MHz oscillator. For example, If we want to generate the output frequency at Q(16) it will be:

$$f_{16} = \frac{100,000,000}{2^{(16+1)}} = 131.072KHz$$

Exercise: Door lock code.

Using the **Basys 3** board for its implementation, simulate opening a door using a secret code. Press three buttons to enter the four-digit code (for example, define the code: 2-0-1-2). The code can be programmed according to the following scheme:

• The first digit will be defined by the switches [7:6], according to the combination “00”, “01” and “10” of the buttons defined for the number “0”, “1” and “2” respectively.

• The second digit will be defined by the switches [5:4].

• The third digit will be defined by the switches [3:2].

• Finally, the fourth digit will be defined by the switches [1:0].

We must press all four digits before knowing if the code was correct. If the code was correct, an LED[0] will light up, otherwise another LED[1] will light up.

Exercise: Rising-edge detector.

Is a circuit that generates a short, one-clock-cycle pulse (tick) when the input signal changes from '0' to '1', known as a rising edge. Is a key technique for identifying signal changes that ensures precise timing and accurate responses in applications like event triggering, timing analysis and signal synchronization.

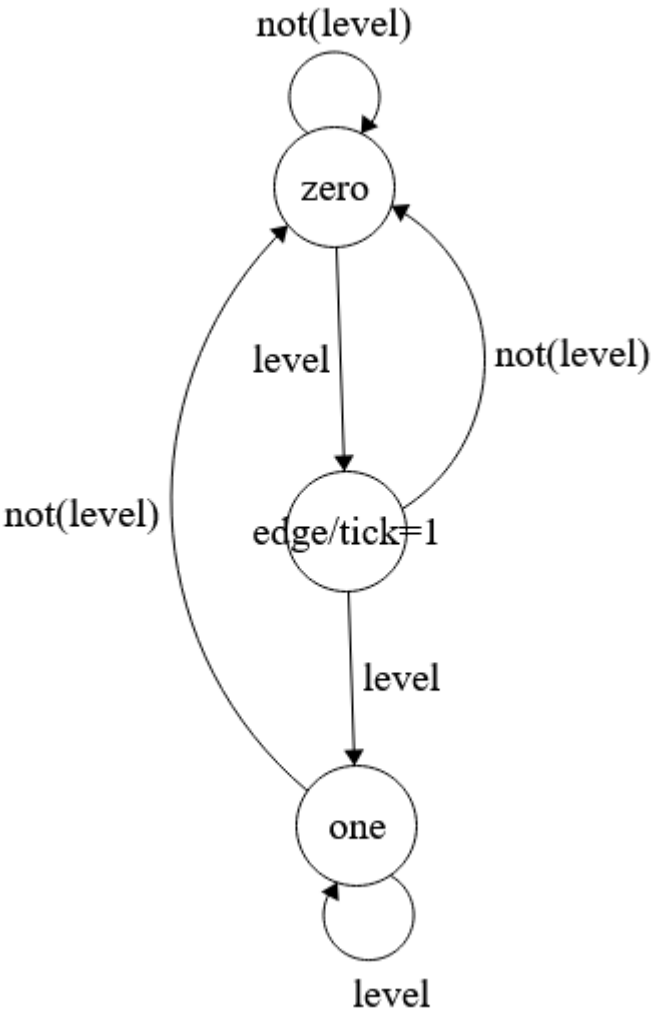


Fig. 2: Moore machine edge detector

Moore machine edge detector

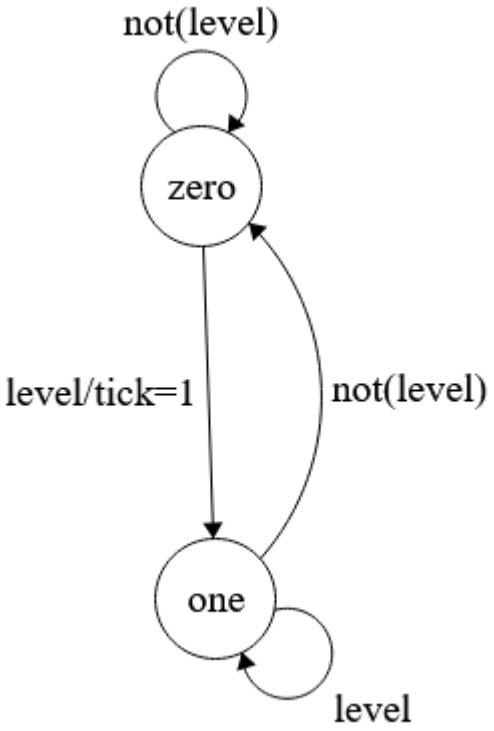


Fig. 3: Mealy machine edge detector

Mealy machine edge detector

Exercise: Debouncing circuit.
When we move a switch, its contact may bounce back and forth a few times before settling down

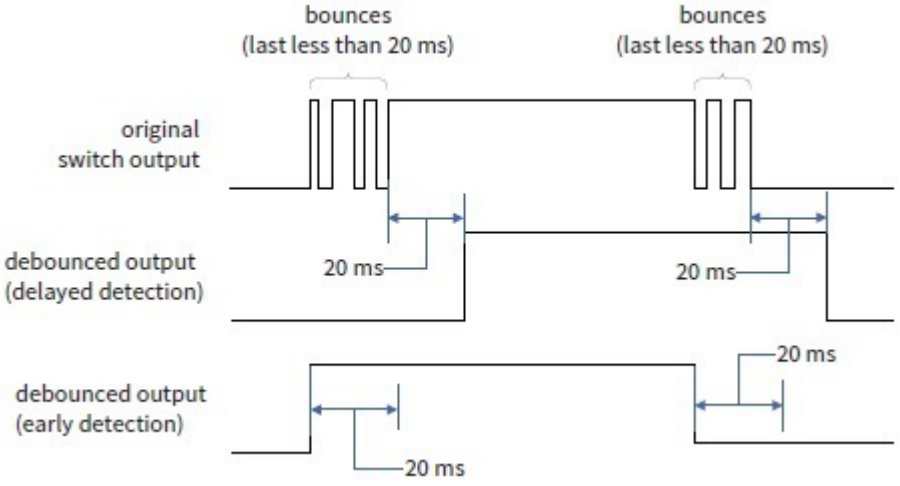


Fig. 4: debounced signal

debounced signal

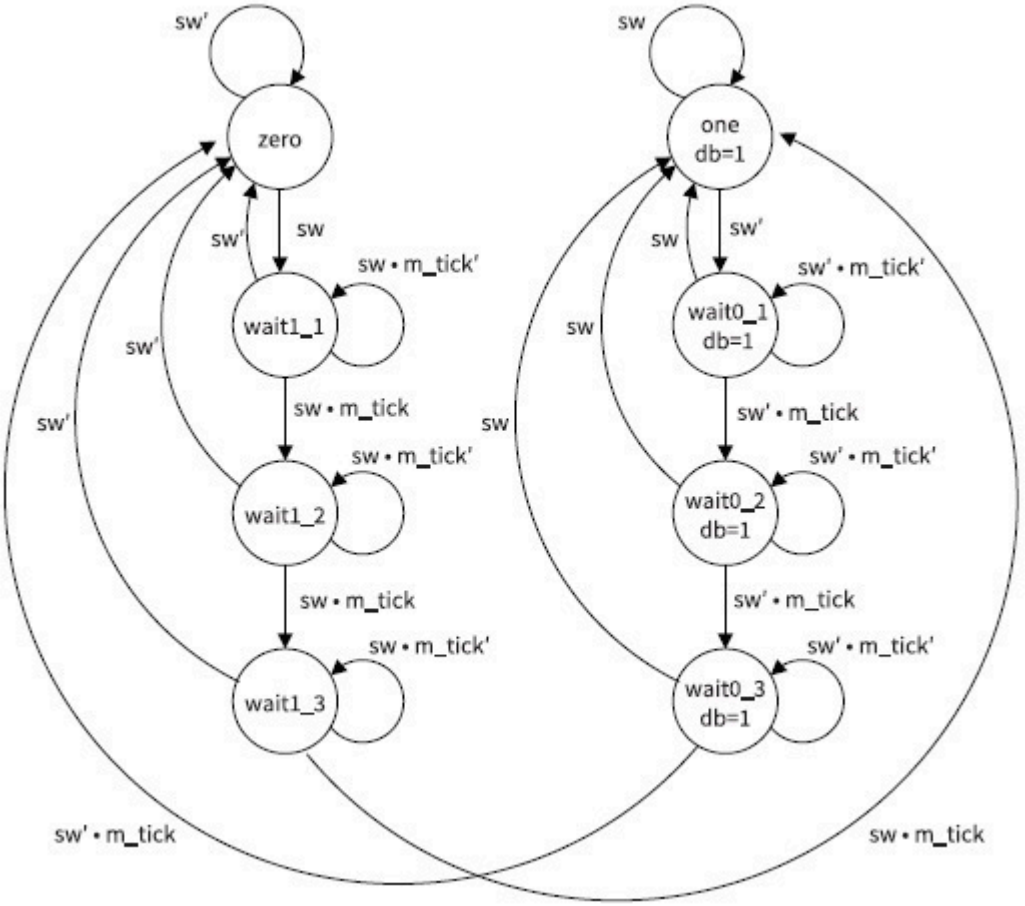


Fig. 5: debouncing (delayed detection) fsm

debouncing (delayed detection) fsm

To verify the operation of the rising-edge detector and the debouncing circuit, we will design a bounce counting circuit.

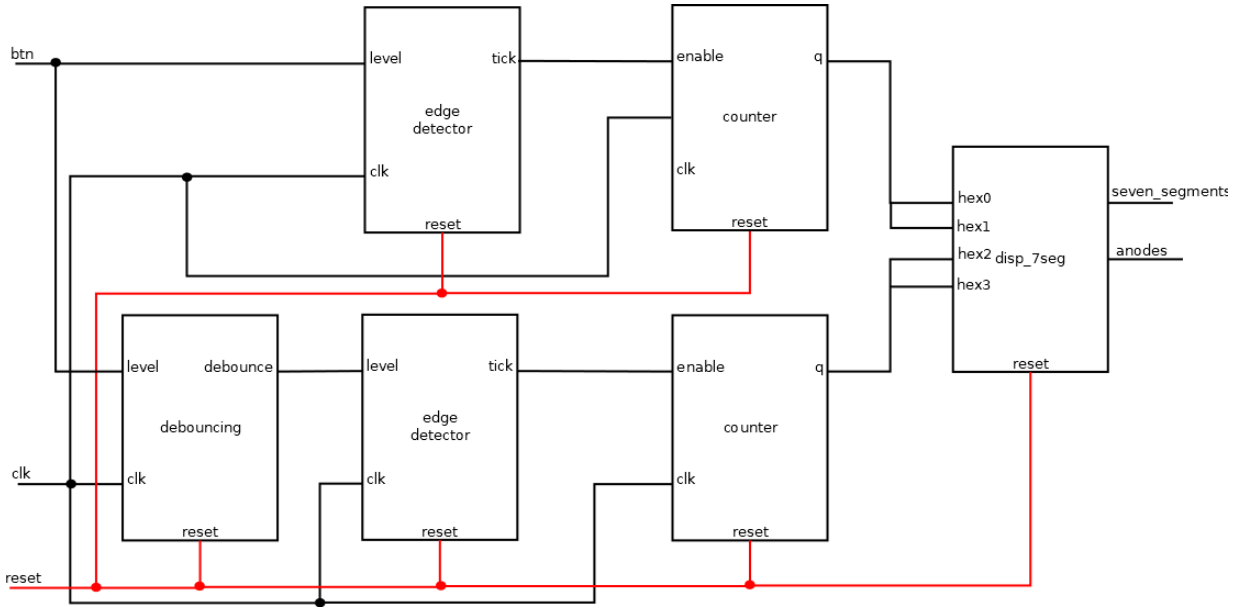


Fig. 6: test debouncing

test debouncing



>jlb/pacheco/